

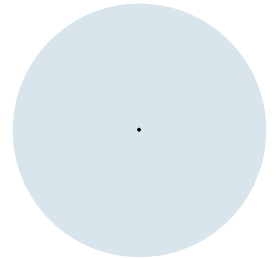
# Einführung in die Programmierung (WIN+BIT)

Klassen I: Attribute, Konstruktoren, Instanz-Methoden und toString

Prof. Dr. Johannes C. Schneider / Prof. Dr. Markus Eiglsperger



# Konstrukturen



# Motivation

- Initialisierung einer Adresse kostet viele Zeilen Code

```
Address beate = new Address();  
beate.name = "Beate Beispiel";  
beate.street = "Lindenstr. 11";  
beate.city = "Konstanz";  
beate.email = "beate@example.org";  
beate.postcode = 78464;  
beate.favorite = true;
```



code12.c\_basic  
Main1

- Geht das nicht einfacher? Vielleicht in einer Anweisung?

```
Address beate = new Address("Beate Beispiel",  
    "Lindenstr. 11", "Konstanz", 78464,  
    "beate@beispiel.org", true);
```

# Parameterloser Konstruktor, automatisch (per default)

- Mechanismus zur Initialisierung einer Klassen-Instanz

```
public class Address {  
    public String name;  
    public String street;  
    public String city;  
    public int postcode;  
    public String email;  
    public boolean favorite;  
}  
Address address = new Address();
```



code12.c\_basic  
Main1

- `new Address()` ruft den implizit vorhandenen Default-Konstruktor `public <Klassenname>()` auf. Dieser Konstruktor hat keine Argumente.

# Parameterloser Konstruktor, explizit

- Man kann diesen argumentlose Konstruktor auch explizit definieren und dort ggf. weitere Anweisungen ausführen:

```
public class Address {  
    :  
  
    public Address() {  
        System.out.println("address initiated");  
    }  
}  
  
Address address = new Address();
```



code12.d\_constructor  
Main1

# Konstruktor mit Parametern

ab hier im Beispiel nur  
noch Straße und Stadt,  
komplettes Beispiel später

- Konstruktor mit Parametern
- Format: <Typ> <Parametername>, komma-getrennt

```
public class Address3 {  
    public String street;  
    public String city;  
  
    public Address3(String street, String city) {  
        this.street = street;  
        this.city = city;  
    }  
}  
  
Address3 address = new Address3("Lindenstraße", "Konstanz");
```



code12.d\_constructor  
AddressSimple

# this

- `this` ist ein versteckter Parameter und enthält eine Referenz auf die Instanz, zu der die ausgeführte Methode/der ausgeführte Konstruktor gehört

## Using the `this` Keyword

Within an instance method or a constructor, `this` is a reference to the *current object* — the object whose method or constructor is being called. You can refer to any member of the current object from within an instance method or a constructor by using `this`.

<http://docs.oracle.com/javase/tutorial/java/javaOO/thiskey.html>

# Konstruktor mit Parametern

- Konstruktor mit Parametern
- Format: <Typ> <Parametername>, komma-getrennt

```
public class Address3 {  
    public String street;  
    public String city;  
  
    public Address3(String street, String city) {  
        this.street = street;  
        this.city = city;  
    }  
}
```

```
Address3 address = new Address3("Lindenstraße", "Konstanz");
```



code12.d\_constructor  
AddressSimple



# Initialer Wert eines Klassenattributs

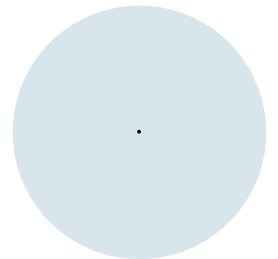
```
public class Address {  
    public String name;  
    public String street;  
    public String city;  
    public int postcode;  
    public boolean favorite;  
}
```

```
Address address = new Address();  
System.out.println(address.favorite); //false  
System.out.println(address.postcode); //0  
System.out.println(address.name); //null
```

Standardwerte nach Initialisierung:

- für Zahlen 0,
- für Variablen vom Typ boolean den Wert false,
- für char das Unicode-Zeichen \u0000 und
- für Referenzvariablen `null`.

# Instanzmethoden



# Klassenmethoden, Wiederholung

- **Klassenmethoden** kann man über den Namen der Klasse direkt aufrufen.
  - Math ist eine Klasse mit vielen Klassenmethoden

```
double randomValue = Math.random() * 10;  
double twoToThePowerOfTen = Math.pow(2.0, 10.0);  
double minimum = Math.min(11.0, 42.0);
```

- Weitere Beispiele: Klassenmethoden von Files und Arrays

```
if (Files.exists(path)) { ... }  
Arrays.sort(myArray);
```

- **Klassenmethoden** nennt man auch **statische Methoden**, weil in der Methodendefinition das Schlüsselwort `static` steht

```
public static double min(double a, double b) { ... }
```

# Instanzmethoden

- In Java gibt es Methoden, die man erst aufrufen kann, wenn man eine Instanz einer Klasse erzeugt hat
  - new erzeugt eine Instanz der Klasse Scanner
  - Hier kann man jetzt die Methode nextLine aufrufen

```
Scanner inputScanner = new Scanner(System.in);  
String input = inputScanner.nextLine();
```

- Das String-Literal "Hallo" erzeugt eine String-Instanz
  - Auf dieser Instanz kann man jetzt die Methode toUpperCase aufrufen

```
String halloUpper = "Hallo".toUpperCase();
```

- Hier jetzt: Eigene Instanzmethoden definieren

# Instanzmethoden

- Methoden einer *Instanz* (eines Objektes) können Bezug auf Attribute und Methoden des Objektes nehmen:

```
public <Rückgabetyt> <Methodenname>(<FormaleParameter>) {  
    this.<Attributname> = 42;  
    this.<MethodenName>(<tatsächliche Parameter>);  
}
```

- Klassenmethoden können das nicht (da gibt es ja im Allgemeinen keine Instanz der Klasse, also kein `this`)

# Beispiel Address.java

```
public class Address {  
    public String name;  
    public String street;  
    public String city;  
    public int postcode;  
  
    public void setAddress(String street, int postcode, String city) {  
        this.street = street;  
        this.postcode = postcode;  
        this.city = city;  
    }  
}
```

Instanzmethode:

- definiert *ohne* static
- benutzt this zum Attributzugriff



code12.e\_methods.  
Address

# Beispiel Address.java

```
public class Address {  
    public String name;  
    public String street;  
    public String city;  
    public int postcode;  
  
    public String postCodeAndCity() {  
        String formatted = this.postcode + " " + this.city;  
        return formatted;  
    }  
  
    public String fullAddress() {  
        String formattedAddress = this.name + ", " + this.street;  
        formattedAddress += ", " + this.postCodeAndCity();  
        return formattedAddress;  
    }  
}
```

Instanzmethode:

- definiert *ohne* static
- benutzt `this` zum Attributzugriff

weitere Instanzmethode:

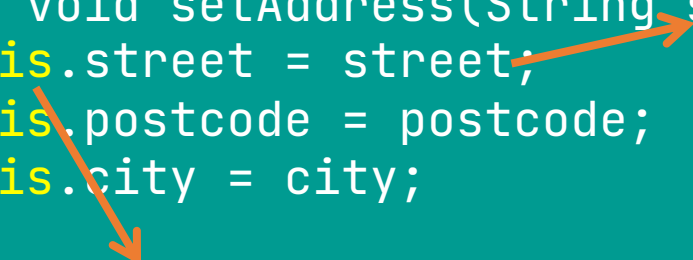
- ruft andere Instanz-Methode auf:  
`this.postCodeAndCity()`



code12.e\_methods.  
Address

# this (Wiederholung)

```
public void setAddress(String street, int postcode, String city) {  
    this.street = street;  
    this.postcode = postcode;  
    this.city = city;  
}  
Address myAddress = new Address(); // Address hat Attribute street,  
                                   // postcode und city  
myAddress.setAddress("Lindenstr", 78462, "Konstanz");
```



- `this` ist ein versteckter Parameter und enthält eine Referenz auf die Instanz, zu der die ausgeführte Methode gehört

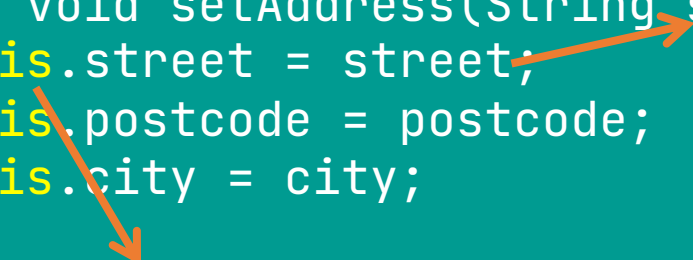
## Using the `this` Keyword

Within an instance method or a constructor, `this` is a reference to the *current object* — the object whose method or constructor is being called. You can refer to any member of the current object from within an instance method or a constructor by using `this`. <http://docs.oracle.com/javase/tutorial/java/javaOO/thiskey.html>



## this = myAddress (im Beispiel)

```
public void setAddress(String street, int postcode, String city) {  
    this.street = street;  
    this.postcode = postcode;  
    this.city = city;  
}  
Address myAddress = new Address(); // Address hat Attribute street,  
                                   // postcode und city  
myAddress.setAddress("Lindenstr", 78462, "Konstanz");
```

An orange arrow originates from the word 'this' in the first code block and points to the variable 'myAddress' in the second code block, illustrating that 'this' refers to the object 'myAddress'.

– mit this = myAddress wird daraus:

```
public void setAddress(String street, int postcode, String city) {  
    myAddress.street = street;  
    myAddress.postcode = postcode;  
    myAddress.city = city;  
}  
Address myAddress = new Address(); // Address hat Attribute street,  
                                   // postcode und city  
myAddress.setAddress("Lindenstr", 78462, "Konstanz");
```

An orange arrow originates from the word 'myAddress' in the second code block and points to the variable 'myAddress' in the third code block, illustrating that the code is now using the variable name directly instead of 'this'.

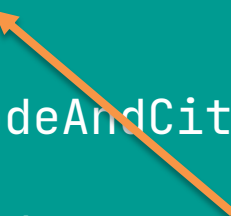
# this kann man weglassen

```
public class Address {  
    public String name;  
    public String street;  
    public String city;  
    public int postcode;  
  
    public String postCodeAndCity() {  
        String formatted = postcode + " " + city;  
        return formatted;  
    }  
  
    public String fullAddress() {  
        String formattedAddress = name + ", " + street;  
        formattedAddress += ", " + postCodeAndCity();  
        return formattedAddress;  
    }  
}
```

- Dieser Code funktioniert auch ohne this
- An den diesen (und den anderen) Stellen weiß der Compiler, dass Attribut und Instanz-Methode verwendet werden soll

# Aufpassen: Verwechslungsgefahr!

```
1 public class Address {  
2     public String name;  
3     public String street;  
4     public String city;  
5     public int postcode;  
6  
7     public String postCodeAndCity() {  
8  
9         String formatted = postcode + " " + city;  
10        return formatted;  
11    }  
12 }
```



this weggelassen: Automatischer  
Verweis auf Attribut

# Aufpassen: Verwechslungsgefahr!

```
1 public class Address {  
2     public String name;  
3     public String street;  
4     public String city;  
5     public int postcode;  
6  
7     public String postCodeAndCity() {  
8         int postcode = 12345;  
9         String formatted = postcode + " " + city;  
10        return formatted;  
11    }  
12 }
```

Aber aufpassen: Neue Variable in Zeile 8 ändert auf einmal das Verhalten der Zeile darunter

# Verwechslungsgefahr verhindern: this verwenden!

```
1 public class Address {  
2     public String name;  
3     public String street;  
4     public String city;  
5     public int postcode;  
6  
7     public String postCodeAndCity() {  
8         int postcode = 12345;  
9         String formatted = this.postcode + " " + this.city;  
10        return formatted;  
11    }  
12 }
```



**Daher Empfehlung: Bei Attributen immer this für den Zugriff verwenden!**

# Zugriff "von außen" auf Attribute und Methoden

– <Instanzname>.<Attributname>

```
address.name = "Beate Beispiel";  
System.out.println(address.city);  
address.postcode = 78462;
```

vorige Vorlesung

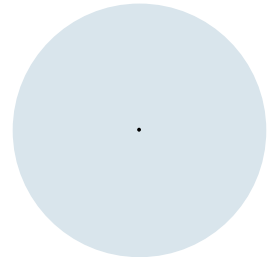
– <Instanzname>.<Methodenname>(<TatsächlicheParameter>)

```
address.setAddress("Alfred-Wachtel-Str.", 78462, "Konstanz");  
System.out.println(address.fullAddress());
```



code12.e\_methods.  
Methods

# **Sichtbarkeit/Kapselung**



# Öffentlicher Zugriff

- Bisher: Alle Klassen und ihre Konstruktoren, Methoden und Attribute sind als `public` deklariert
- Man kann von überall darauf zugreifen
- Man kann interne Implementierungsdetails sehen und verändern!
- Das ist üblicherweise **nicht erwünscht!**



# Öffentlichen Zugriff minimieren

- Alle Klassen und ihre Methoden und Attribute öffentlich zugänglich zu machen, ist kein guter Programmierstil:
  - Interne Daten und Implementierungsdetails sollten verborgen sein
  - Interaktion mit Klasse sollte nur über bestimmte Konstrukturen und Methoden möglich sein
  - Hilfsklassen sollten nur innerhalb eines Packages zugreifbar sein

# Kapselung

- Interne Implementierungsdetails werden nach außen hin verborgen ("gekapselt")
  - z.B. bei Attributen über getter- und setter-Methoden
- Nur das absolut notwendige wird als öffentlich deklariert
  - Methode oder Konstruktor zum Erzeugen einer Instanz
  - Schnittstellen-Methoden, über die man mit den Objekten interagieren soll
- Prinzip nicht neu (d.h. lange vor Java)
  - Modularisierung, Komponentenbildung, Programmbibliotheken ...

---

## On the Criteria To Be Used in Decomposing Systems into Modules

D.L. Parnas  
Carnegie-Mellon University

CACM Dez. 1972

# Kapselung – wozu?

Vorteil:

- Die Implementierung kann lokal geändert werden, ohne dass global alle Stellen, bei denen eine Klasse benutzt wird, angepasst werden müssen

Anwendungsfälle:

- Ersatz einer Implementierung durch eine effizientere
- Ersatz zugekaufter Komponenten
- Entwicklung im Team

# Kapselung durch Verstecken

- Variablen und interne Methoden verstecken
- Zugriff auf Objekte nur über öffentliche Methoden
- Zugriffsrechte auf eine Klasse
  - durch eigene Klasse ("private")
  - durch abgeleitete Klassen ("protected") - später
  - durch Klassen desselben Packages ("package-private" – default)
  - durch beliebige Klassen ("public")



# Regel 1: Attribute verstecken

code12.f\_gettersetter.  
Address

- Attribute sollten in der Regel als `private` deklariert werden
  - Dann kann nur innerhalb der Klasse selbst darauf zugegriffen werden
- Zugriff auf Variable über  
`getAttribut() {`  
    `return this.attribut;`  
`} // Getter-Methode`  
  
`setAttribut(Typ neuerWert) {`  
    `this.attribut = neuerWert;`  
`} // Setter-Methode`

```
public class Address {  
  
    private int plz;  
    // weitere Attribute...  
  
    public int getPlz() {  
        return this.plz;  
    }  
  
    public void setPlz(int plz) {  
        this.plz = plz;  
    }  
}
```



# Vorteile Getter/Setter

code12.e\_methods.  
Methods

- Validierung beim Setzen möglich
- nur getter, keine setter: Attribute können nicht mehr verändert werden
- Variablentyp und -nutzung kann innerhalb der Klasse anders implementiert werden
  - z.B. Attribut intern als int gespeichert, aber als String nach außen gegeben
- Schnittstelle nach außen bleibt gleich

Das Thema Exceptions behandeln wir später. Hier nur so viel: Das Programm wird dadurch mit einer Fehlermeldung beendet.

```
public class Address {  
  
    private int plz;  
    // weitere Attribute...  
  
    public int getPlz() {  
        return this.plz;  
    }  
  
    public void setPlz(int plz) {  
        if (plz < 0 || plz > 99999) {  
            throw new  
IllegalArgumentException("wrong plz");  
        }  
        this.plz = plz;  
    }  
}
```

# Regel 2: Interne Methoden verstecken

- Methoden
  - nur innerhalb der Klasse benutzt: `private`
  - auch von Subklassen benutzt: `protected`
  - von "außen" benutzt: `public`

```
public class Person {  
  
    private String email;  
    // interne Hilfsmethode  
    private boolean emailCorrect(String email){  
        return email.contains("@");  
    }  
    // von "außen" benutzte Methode  
    public void setEmail(String email) {  
        if (!emailCorrect(email)) {  
            throw new IllegalArgumentException();  
        }  
        this.email = email;  
    }  
    //...  
}
```

# Übliche Regeln/Empfehlungen zusammengefasst (Vorläufige Version)

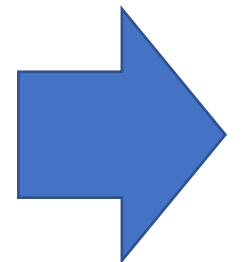
- **Klassen**
  - üblicherweise `public`
- **Attribute**
  - immer `private` (Zugriff über öffentliche get- und set-Methoden, wenn gewünscht)
- **Konstruktoren**
  - üblicherweise `public`
- **Methoden**
  - nur innerhalb der Klasse benutzt: `private`
  - von "außen" benutzt: `public`



# Zugriffsmodifikatoren (vereinfacht)

|                              | <i>eigene<br/>Klasse</i> | <i>Klassen<br/>desselben<br/>Pakets</i> | <i>alle anderen<br/>Klassen</i> |
|------------------------------|--------------------------|-----------------------------------------|---------------------------------|
| <b>public</b>                | +                        | +                                       | +                               |
|                              |                          |                                         |                                 |
| <i>kein<br/>Modifizierer</i> | +                        | +                                       | -                               |
| <b>private</b>               | +                        | -                                       | -                               |

Auf den folgenden  
Folien sind alle Fälle  
noch einmal detailliert  
(und korrekt) aufgeführt.



# Zugriffsmodifikatoren, public

- **Klassen**, die mit **public** gekennzeichnet sind, können überall (auch in anderen Paketen) genutzt werden\*.
- Auf **Attribute**, **Methoden** und **Konstruktoren**, die als **public** vereinbart sind, kann überall dort zugegriffen werden, wo auf die Klasse zugegriffen werden kann.

# Zugriffsmodifikatoren default, private

- **default ("package-private")**: Sind **weder public noch private** angegeben, so können die Klassen, Attribute, Methoden und Konstruktoren in allen Klassen desselben Pakets benutzt werden.
- **Attribute, Methoden und Konstruktoren**, die mit **private** gekennzeichnet sind, können **nur in der eigenen Klasse** genutzt werden.